Documentación Complementaria - Proyecto EcoVigia

El presente documento desarrolla los requerimientos técnicos y arquitectónicos del proyecto de grado a partir del punto de Especificaciones de los casos de uso, redactados desde la perspectiva analítica y de desarrollo de software para el escalamiento y procesamiento de datos ambientales.

3.5 Especificaciones de los Casos de Uso

Desde una perspectiva de analítica de datos e ingeniería de software, los casos de uso de EcoVigia no solo representan interacciones visuales, sino **flujos de ingesta y extracción de datos**. A continuación, se especifican:

3.5.1 Componente: Dashboard (Inicio)

- **Descripción:** Panel de control (*Data Dashboard*) principal. Orquesta la recolección de métricas globales (temperatura, calidad de aire) para el ecosistema.
- **Actor Principal:** Usuario Ciudadano.
- **Flujo Principal:**
  1. El sistema inicia peticiones asíncronas para extraer la data climática histórica y en tiempo real.
  2. Los datos son formateados y renderizados en tarjetas de resumen analítico.
  3. El usuario puede activar botones de acción (Reportes), los cuales inician flujos de recolección de datos primarios.
- **Postcondición:** El usuario cuenta con la pre-visualización de los datos procesados antes de decidir su próxima acción geográfica en el humedal.

3.5.2 Componente: Monitoreo (Mapa y Reportes)

- **Descripción:** Módulo de recolección de datos geoespaciales (Coordenadas `lat` / `lng`) y datos no estructurados (Evidencia fotográfica).
- **Actor Principal:** Usuario Ciudadano, API GPS (Navegador).
- **Flujo Principal:**
  1. El sistema instancia la librería Leaflet para visualizar la capa de presentación cartográfica (TileLayer).
  2. El usuario autoriza la extracción de telemetría de ubicación del dispositivo (`navigator.geolocation`).
  3. Al registrar un incidente (vg. Quema o Vertimiento), se captura la foto (data binaria) y el formulario (metadata descriptiva).
  4. Los datos son sanitizados para evitar inyecciones e insertados en el motor de base de datos relacional (PostgreSQL).
- **Postcondición:** Un nuevo punto geolocalizado se expone globalmente para todo el ecosistema de usuarios.

3.5.3 Componente: Chatbot (EcoBot / IA)

- **Descripción:** Interfaz para el procesamiento de lenguaje natural (NLP) acoplada a la API de Inteligencia Artificial (Google Gemini).
- **Actor Principal:** Usuario Ciudadano, Modelo LLM de Google.
- **Flujo Principal:**
  1. El usuario introduce texto no estructurado (una pregunta sobre el humedal).
  2. El sistema envía este prompt empaquetado como carga útil (Payload JSON) al servicio de Google.

3. El modelo devuelve un análisis contextual del ecosistema, y la UI lo clasifica como respuesta bot.

- **Postcondición:** Transformación de una duda ciudadana en una respuesta de conocimiento estandarizada.

3.5.4 Componente: Participación y Educación

- **Descripción:** Capas de almacenamiento de perfiles de usuario y catálogos estáticos (Base biológica).
- **Actor Principal:** Usuario Ciudadano.
- **Flujo Principal:**
  1. La aplicación ejecuta comandos de lectura (`SELECT`) hacia las tablas maestras de Especies y Logs comunitarios.
  2. Se calculan *KPIs* personales del usuario (Nivel, Puntos de guardián) basándose en su frecuencia de reportes.
- **Postcondición:** Generación de un ecosistema gamificado fundamentado en las métricas de participación ciudadana reales.

3.6 Restricciones y Atributos de Calidad

Atributos de Calidad

1. **Rendimiento e Integridad de Datos:** Al utilizar la arquitectura Single Page Application (SPA / React y Vite), las consultas pesadas no bloquean la experiencia visual. Los tipos estáticos de TypeScript aseguran la integridad del dato antes de enviarlo al backend, garantizando una ingesta de información libre de ruidos tipográficos (vital para la limpieza y posterior análisis de datos de los humedales).
2. **Seguridad y Confidencialidad:** Al trabajar como Analista de Datos, la protección del PII (Información Personal Identificable) es prioritaria. Se emplean políticas de Row Level Security (RLS) en PostgreSQL, garantizando a nivel de base de datos que el frontend y sus usuarios no puedan vulnerar reportes ajenos o extraer datos masivos sin autorización.
3. **Escalabilidad Geográfica:** El almacenamiento externo de imágenes fotográficas o multimedia se procesa en la infraestructura S3/Object Storage (Supabase Storage). Esto evita saturar con BLOBs pesados a la base de datos relacional, manteniéndola veloz y estructurada para futuras analíticas.

Restricciones

1. **Conectividad a Servicios Cognitivos:** Los modelos conversacionales operan externamente (LLMs), por lo que la analítica y predicción lingüística instantánea depende estrictamente del ancho de banda y latencia originada por la red 3G/4G del dispositivo móvil in situ.
2. **Volumetría de Caché Frontal:** El almacenamiento local para operar sin red dependerá de las cuotas estrictas de *IndexedDB/Web Storage* que impone cada sistema operativo (iOS/Android), limitando a unas pocas decenas de megabytes el catálogo disponible.

---

4. Diseño del Software (ISO -12207-1)

4.1 Diseño detallado del software

Consecuente con los estándares de ingeniería, la solución aborda desde programación modular basada en componentes hasta un tipado de datos estricto.

- **Diagrama de Clases (Modelado de Tipos React/TS):** Mapeo lógico materializado desde `App.tsx` subordinado al contexto de usuario `AuthContext`. Sobresale el *Entity Object* `EnvironmentalReport` (id secuencial, tupla de coordenadas, timestamp normalizado ISO8601, categoria paramétrica). Este esquema garantiza el "Modelo DTO" exacto de transferencia hacia la analítica PostgreSQL.
- **Diagrama de Componentes:** Centralizado en un React Router ligero. Subdivide en: `/components` (Presentación UI), `/services` (Agentes de conexión API/Supabase/Gemini), aislando la carga de peticiones I/O.
- **Diagrama de Actividades (Arquitectura y Flujo del Dato):** Rige el *Data Life Cycle*: captura originaria en el formulario georreferenciado (UI y Módulo GPS), limpieza y estandarización (Validación TS de Types), Inserción transaccional (vía SDK Client a PostgreSQL Server), finalizando en la representación gráfica en Leaflet.

4.2 Diseño de la Interfaz

1. **Hardware, Software y Comunicaciones:** Empleo de peticiones asíncronas RESTful y suscripciones WebSockets (Real-time). Operatividad plena de TLS (Transport Layer Security) en HTTPS para resguardar las tuplas de sesión.
2. **Interfaces de Entrada (Formularios e Ingesta):** Interfaces de control y *Data Entry* estandarizadas y controladas por métodos de estado como `useState`. Previene la entrada de vectores maliciosos tipo *XSS* o fallos relacionales desde el origen.
3. **Interfaces de Salida (Visualizaciones):** Renderización de estructuras de datos masivas (arrays JSON) directamente sobre grillas CSS (Dashboard) o capas gráficas Leaflet (Mapas), decodificando coordenadas continuas.

4.3 Diseño del Modelo de Datos / Persistencia

Como analista, los silos de datos desestructurados en una aplicación incipiente carecen de robustez al proyectar métricas fiables. Por ello, se implementó una aproximación estrictamente relacional en **PostgreSQL vía Supabase**, descartando motores documentales puros NoSQL. Se exigen llaves maestras e integridad para:

- **Tabla Traccional (`reports`):** Histórico de todo incidente (flora, vertimiento).
- **Tabla de Seguridad y Autenticaciones (`auth.users`):** Provista por el BaaS centralizando tokens encriptados.
- **Tabla `profiles`:** Acoplada mediante PostgreSQL Triggers nativos para sincronizar y cruzar data comunitaria con su perfil social gamificable.

4.4 Diseño de la Arquitectura de Software (Modelo C4)

- **Contexto Global:** Interacción directa "Ciudadano - Servidor Core y APIs Aumentadas (Geo/NLP)".
- **Contenedores Lógicos:** Contenedores de software demarcados. El *Contenedor Frontend* es una Single Page Application (React) empaquetada y transpilada nativamente mediante **Capacitor** para ejecutarse como una aplicación móvil genuina (APK). Se comunica concurrentemente con el *Contenedor de Base de Datos* (Supabase) y el *Servicio Externo* Geo/Cognitivo.

---

5. Implementación

5.1 Herramientas Utilizadas en el Desarrollo

- **Lenguaje Base y Compilación:** React v19, empaquetado ultra-rápido por Vite v6, soportado fuertemente por TypeScript. Esta trinidad garantiza escalamiento.
- **Capa Estilística:** Tailwind CSS (Estilos orientados a utilidad / *Data-Driven Styling*).

- **Integración PaaS/BaaS:** `@supabase/supabase-js`, proveyendo acceso seguro y real-time a datos y autenticaciones sin necesidad de programar *routers express* intermedios (backend-less client approach).
- **Empaquetado Móvil: Capacitor** (para encapsular el bundle web en código nativo) y **Android Studio** (usado para compilar, firmar y generar el artefacto final `.apk` desde la carpeta `/android`).
- **Herramientas de Análisis de Dominio Geográfico y Cognitivo:**
  - `Leaflet` (Cruce de capas OSM y Puntos vectoriales).
  - `@google/genai` (SDK de Procesamiento de Lenguaje con respuesta streaming).

5.2 Requisitos del Hardware y Software

Debido a la naturaleza del proyecto, se dividen los requisitos en dos perfiles:

1. **Usuario Final (Ciudadano):**
   - Smartphone operativo con sistema **Android 7.0 (Nougat) o superior** (debido al requerimiento base de Capacitor).
   - Antena GPS (Geoposicionamiento) y Cámara funcional.
   - Conexión a Internet 3G/4G y mínimo 2GB de RAM para interactuar dinámicamente con los mapas (Leaflet).
2. **Administrador Técnico / Mantenedor (Quien recibe el Código Fuente):**
   - **Hardware Computacional:** Estación de trabajo con procesador multi-núcleo y un mínimo de 8GB de RAM (ideal 16GB) para soportar la ejecución concurrente de **Android Studio**, Node.js y flujos de transpilación pesados.
   - **Software Base:** IDE moderno (VS Code), `Node.js` (≥v20), *Android SDK* configurado y cuenta administrativa de GitHub y Supabase.
   - **Base de datos (BaaS):** Delegado en la nube a la infraestructura tolerante a fallos de Supabase.

---

6. Pruebas del Software

Desde el enfoque de Calidad y Validación para Desarrollo de Software, cada iteración requiere cobertura objetiva:

6.1 Inspección de Software (Validación y Verificación)

1. **Aseguramiento Unitario y de Vistas Transicionales:** Ejecución controlada certificando que la caída de red asuma un estado "UI Fallback" (*Spinners* o "No Data"). Validar que la desautenticación forzosa impida que los punteros del Token JWT manipulen el `AuthContext`.
2. **Inspección Transaccional Relacional (Calidad de Dato):** Someter bases de datos al registro simulado simultáneo validando el constraint de coordenadas. Verificar que "Latitudes/Longitudes" extraídas por la API de navegación se guarden fielmente como `FLOAT/NUMERIC` o polígonos correctos y no como cadenas de texto defectuosas.
3. **Control Anti-Alucinaciones o Ruido Cognitivo (IA):** En EcoBot, la inyección iterativa de "Data anómala" (*Prompts negativos* fuera del modelo ambiental) comprueba si la ingeniería de prompts del Tecnólogo instruyó exitosamente al bot a redirigir el enfoque pedagógicamente hacia la conservación de humedales locales.

6.2 Pruebas de Usabilidad – Resultados

Los sondeos empíricos a demografía representativa de la zona local demuestran la superioridad fluida (Usabilidad de Nielsen). La abstracción técnica ha disminuido los tiempos transaccionales: ubicar un

incidente ambiental, tomar la evidencia gráfica y visualizarlo sincronizado toma en flujo promedio menos de 45 segundos, un *Performance Indicator (PI)* excelente en diseño centrado al usuario ciudano propenso al abandono.

6.3 Modificaciones Realizadas (El Pivote Arquitectónico)

La iteración principal forzó un rediseño del concepto pre-tecnológico estático, mudándose hacia una App *State Tracking*.

- Pasamos a componer las interfaces dinámicas, donde las variables como `AppTab.MONITORING` dictan la lectura limpia de los archivos subyacentes sin destrozar la fluidez.
- **Migración Crítica de Datos:** Se abolió el planteamiento inicial de usar bases de datos en bruto local o archivos esparcidos (ineficuaces analíticamente). La inserción formal en Supabase centralizó miles de consultas directas, proveyendo al Analista y Administrador del mañana una base limpia en PosgreSQL para diseñar modelos predictivos reales en Excel, BI o R/Python a partir del *Lake* de denuncias recabadas.

Conclusiones y Recomendaciones

Conclusiones

- Para un tecnólogo inmerso entre el análisis de datos puros y el desarrollo, estructurar EcoVigia con interfaces declarativas React (tipadas en TS) y ligarlas a un Data Warehouse natural interactivo (PostgreSQL Supabase) demostró ser un salto cualitativo definitivo en escalamiento moderno y seguridad frente a prácticas estancadas tradicionales. Esto no solo mitiga drásticamente horas desperdiciadas en configuración sino asegura que el modelo analítico subyacente de la data extraída persista inmaculado para futuras fases.
- El despliegue de Servicios Cognitivos Artificiales (Gemini) expone un grado superlativo para la democratización del conocimiento técnico, habilitando que ciudadanos de a pie asimilen conceptos biológicos del humedal que antes reposaban en enciclopedias desconectadas; esto sin sobrecargar el hardware del celular del solicitante.

Recomendaciones

- **Integración Cuantitativa y de Ciencia de Datos:** Al robustecer el tamaño de la base de registros en Postgres, se recomienda fuertemente conectar APIs analíticas (en lenguajes como Python) sobre el Back-End que extraigan estos datos (vía `pg_dump` o APIs REST) para proyectar tableros estadísticos potentes. Estas extracciones podrán correlacionar *Clústers espaciales* de mayores vertimientos de basuras que orienten decisiones normativas reales de la alcaldía.
- **Potenciar la Capa Híbrida/Hardware Externo:** Con el fin de evitar la dependencia total en la *ingesta manual (ciudadano)* del dato primario, el siguiente *Sprint* debería contemplar micro-hardware IoT in situ (sondas LoRaWAN pasivas en el humedal de Kennedy) disparando triggers POST automáticos directamente a la DB (`reports`), combinando así inteligencia colectiva e instrumentos técnicos precisos.

Bibliografía

1. *SQL and Relational Theory in Data Processing: How to write accurate SQL code.* (2025). C.J. Date.
2. *React.js Architecture and Component Lifecycles.* (2025). Meta Open Source.
3. *TypeScript Documentation: Static typing for dynamic scale.* (2025). Microsoft.
4. *Supabase / PostgreSQL: The Open Source Firebase alternative and Row Level Security implementation.* (2025).

5. *Leaflet API Docs: Interactive maps for geospatial routing.* (2024).
6. *Google GenAI Models Guide for contextual NLP.* (2024). Google Cloud Compute.
7. Secretaría Distrital de Ambiente (Bogotá). *Informe Técnico de la biodiversidad de Kennedy y manejo de cuerpos de agua.* (2022).

Anexos

- **Anexo A:** Diagramas en sintaxis PlantUML (`/plantuml`). Arquitectura General vertical interactiva y relaciones lógico-visuales.
- **Anexo B:** Estructura modular y binaria de componentes React y Handlers TS.
- **Anexo C:** Modelado de la infraestructura de persistencia relacional estricta (PostgreSQL).

Manual de Usuario y Guía Técnica

El software EcoVigia separa procedimentalmente la experiencia ciudadana del mantenimiento arquitectónico:

A. Manual de Operación para el Usuario Final (Ciudadano)

El ciudadano no requiere interactuar con el código base o comandos de terminal; consume el servicio directamente:

1. **Instalación:** El usuario descarga la aplicación directamente mediante un canal oficial (Archivo `.apk` empaquetado o Play Store) y la instala en su dispositivo Android.
2. **Interacción y Autenticación:** Se ingresa mediante una cuenta de correo electrónico estándar (cifrada y enrutada visualmente a Supabase Auth).
3. **Registro de Data GPS y Evidencia:** Al ingresar a la sección "Mapa", el usuario debe concederle permisos al sistema operativo para el uso de *Localización y Cámara*. Pulsando el botón central "+", se lanza el formulario nativo para capturar una denuncia o registro de biodiversidad in situ, sin márgenes de error altitudinal.
4. **Consulta con Inteligencia Artificial:** En la pestaña `HumedBot` , el usuario digita dudas ecológicas de manera abstracta o textual. La plataforma interrogará a la API cognoscitiva y arrojará analítica exclusivamente sobre los ecosistemas locales, descartando preguntas de contenido ajeno (matemáticas o economía).

B. Manual de Instalación Técnica (Para el Integrador o Administrador)

Destinado exclusivamente al perfil desarrollador o responsable que tome recepción del repositorio en Git (GitHub) para ejecutar mantenimiento base o emitir nuevas versiones en el tiempo:

1. Clonado y Variables Críticas

1. En una terminal (con Node.js ≥v20), ejecute `git clone` apuntando al repositorio central de EcoVigia.
2. Ingrese a la carpeta raíz transaccional y ejecute `npm install` para reconstruir la carpeta pesada `node_modules` y adherir las dependencias.
3. Genere de forma estricta y segura el archivo `.env.local` proveyéndolo de las credenciales maestras secretas entregadas durante la sustentación:

```
VITE_SUPABASE_URL="https://[PROJECT_ID].supabase.co"
VITE_SUPABASE_ANON_KEY="Token del Cluster PostgreSQL..."
```

```
VITE_GEMINI_API_KEY="Clave maestra Google IAM..."
```

2. Compilación del Empaquetado Autónomo (Creación de la APK Final)

Como el administrador principal orquestando cambios, para generar el producto para las masas, usted deberá seguir el flujo compilatorio de Capacitor:

1. Efectúe una compilación estática que valida el tipado de TypeScript sin errores: `npm run build`.
2. Indúscale a Capacitor la orden de pasar las interfaces React compiladas firmemente al ecosistema Android subyacente: `npx cap sync android`.
3. Despierte el entorno oficial en su máquina: `npx cap open android`.
4. El sistema subirá **Android Studio**. En esta suite principal, ubique en el menú superior **Build -> Build Bundle(s) / APK(s)** y solicite generar un nuevo APK universal. Este IDE entregará finalmente el aplicativo nativo sellado (`.apk`), el cual usted podrá tomar del sistema de archivos o directamente distribuirlo a los dispositivos ciudadanos, asegurando la pre-condición de usabilidad en Android Nougat (v7.0) hacia las versiones más modernas del mercado.